Student Name: Shiva Sai Krishna S **Register Number:** 1911260318

Course: CSA0421 - Operating System **GitHub Repo:** krishshiva721/CSA04--Assignment1

TASK 3: OS KERNEL STRUCTURES

Architectural Trade-offs: Microkernel vs. Monolithic vs. Modular

1. EVALUATION OF KERNEL ARCHITECTURAL STRUCTURES

Structure	Reliability & Fault Isolation	Performance / IPC Latency	Target Environment Fit
Monolithic (e.g., Traditional Unix)	Low isolation. A single driver crash triggers kernel panic.	Maximum performance. In-kernel function calls without context switches.	Legacy desktop computing. Unsuited for critical patient safety.
Microkernel (e.g., QNX, seL4)	High isolation. Drivers run in user space; crashes restart isolated.	Slight IPC overhead due to message passing and context switches.	**Bedside Patient Monitors.** Guarantees alarm process stays alive even if network driver crashes.
Modular LKM (e.g., Modern Linux)	Moderate isolation. Loadable kernel modules execute in ring 0.	High execution speed with dynamic adaptability.	**Central Database Server.** Allows dynamic module loading (SAN drivers) without server reboot.
Hybrid (e.g., Windows NT, macOS)	Balanced. Executive services share space for performance.	High performance with moderate modular structure.	Doctors' Workstations and Clinical Tablets (Android/Linux hybrid).

2. STRUCTURE DEFEND FOR BEDSIDE MONITORS VS. DATABASE SERVER

Bedside Monitors (Microkernel Defense): Operating in ring 0 is restricted to minimal primitives: IPC, thread scheduling, and basic memory mapping. Device drivers and protocol stacks run in unprivileged user space. If a sensor driver faults, the microkernel restarts the user-space driver process in under 5 ms without interrupting cardiac alarm threads.

Database Server (Modular Monolithic Defense): High-throughput database transactions require direct, low-latency interactions between virtual memory management, page caching, and NVMe disk controllers. A modular kernel provides peak raw performance while allowing storage modules to be upgraded dynamically.

3. ALTERNATIVE CONCEPT & COMPLEXITY ANALYSIS

> **Alternative Scenario: Monolithic Kernel on Bedside Monitors**
>
> If patient monitors run a monolithic kernel instead of a microkernel:
>
> - **Fault Cascade Risk:** A null-pointer dereference in a Wi-Fi driver crashes the entire kernel instantly, causing total monitor blackout and missed cardiac events.
> - **Time Complexity of Service Request:** Monolithic direct function call $T_{call} = O(1)$ (< 10 ns switch) vs. Microkernel IPC message passing $T_{ipc} = O(1)$ with 2 context switches (~1.5 µs).
> - **Memory Space Footprint:** Monolithic kernel retains 150 MB+ in kernel space. Microkernel requires only ~128 KB kernel space overhead, maximizing deterministic static cache allocation for alarm loops.